

Security Scan Report

Target: C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab

Scan Date: May 08, 2026 at 02:37 UTC

Scan ID: 4959f563-beda-407d-9e79-2ba2e25dc376

0 CRITICAL

2 HIGH

9 MEDIUM

2 LOW

Executive Summary

Security scan of 'C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab' identified 38 raw findings, reduced to 13 unique issues after deduplication. ■ 2 HIGH-severity issue(s) should be prioritized for remediation within the next 48 hours. 9 MEDIUM-priority issue(s) should be addressed in the current development sprint. 2 LOW-priority issue(s) have been logged for backlog tracking.

Scan Statistics

Metric	Value
Total Raw Findings	38
After Deduplication	13
Critical Findings	0
High Findings	2
Medium Findings	9
Low Findings	2
Scan Duration	17.0s

Detailed Findings

[HIGH] #1: Insecure Deserialization in C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:282

Risk Score	0.6076
Affected Location	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:282
Vulnerability Type	INSECURE_DESERIALIZATION
CWE ID	CWE-502: Deserialization of Untrusted Data
Detected By	semgrep

Explanation
Insecure deserialization occurs when untrusted data is deserialized without validation, potentially allowing attackers to execute arbitrary code or manipulate application logic.

Code or HTTP Excerpt

```
280 |         try:
281 |             decoded = base64.b64decode(payload)
>>> 282 |             session_object = pickle.loads(decoded)
283 |             return jsonify({"status": "restored", "object": repr(session_object)})
284 |         except Exception as exc:
```

Business Impact
An attacker could execute arbitrary code on the server or manipulate application state to bypass authentication or authorization.

Fix Recommendation
Rule-based recommendation: Validate and whitelist deserialized types. Avoid using pickle or yaml.load() with untrusted input. Use yaml.safe_load() or JSON for data interchange. Implement integrity checks on serialized data.

LLM recommendation: Validate and whitelist deserialized types. Avoid using pickle or yaml.load() with untrusted input. Use yaml.safe_load() or JSON for data interchange. Implement integrity checks on serialized data.

[MEDIUM] #2: Unknown in C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:70

Risk Score	0.5390
Affected Location	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:70
Vulnerability Type	UNKNOWN
CWE ID	CWE-704: Incorrect Type Conversion or Cast
Detected By	semgrep

Explanation
A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
68 |         email = request.form.get("email", "")
69 |         password = request.form.get("password", "")
>>> 70 |         query = f"SELECT * FROM users WHERE email = '{email}' AND password = '{password}'"
71 |         try:
72 |             conn = connect()
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[HIGH] #3: Sql Injection in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:68

Risk Score	0.5170
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:68
Vulnerability Type	SQL_INJECTION
CWE ID	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Detected By	semgrep

Explanation

SQL Injection occurs when user-controlled input is directly concatenated into SQL queries, allowing an attacker to manipulate the query logic. This can lead to unauthorized data access, modification, or deletion of database records.

Code or HTTP Excerpt

```
66 |         error = None
67 |         if request.method == "POST":
>>> 68 |             email = request.form.get("email", "")
69 |             password = request.form.get("password", "")
70 |             query = f"SELECT * FROM users WHERE email = '{email}' AND password = '{password}'"
```

Business Impact

An attacker could read, modify, or delete all data in the database, potentially exposing customer records and sensitive business data.

Fix Recommendation

Rule-based recommendation: Use parameterized queries instead of string concatenation. In Python, use placeholder syntax: `cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,))`. For ORMs, use the built-in query builder instead of raw SQL.

LLM recommendation: Use parameterized queries instead of string concatenation. In Python, use placeholder syntax: `cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,))`. For ORMs, use the built-in query builder instead of raw SQL.

[MEDIUM] #4: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\admin.html:22

Risk Score	0.4747
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\admin.html:22
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation

Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```
20 |         <button class="btn primary" type="submit">Run</button>
21 |     </form>
>>> 22 |     <pre class="console">{{ command_output|safe }}</pre>
23 | </div>
24 | <div class="card">
```

Business Impact

An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[MEDIUM] #5: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:10

Risk Score	0.4508
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:10
Vulnerability Type	UNKNOWN
CWE ID	CWE-611: Improper Restriction of XML External Entity Reference
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
8 | import subprocess
9 | import urllib.request
```

```
>>> 10 | import xml.etree.ElementTree as ET
      11 | from pathlib import Path
      12 | from urllib.parse import unquote, urlparse
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[MEDIUM] #6: Open Redirect in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:87

Risk Score	0.4508
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:87
Vulnerability Type	OPEN_REDIRECT
CWE ID	CWE-601: URL Redirection to Untrusted Site ('Open Redirect')
Detected By	semgrep

Explanation

Open redirect vulnerabilities allow attackers to redirect users to malicious websites by manipulating URL parameters, which can be used for phishing attacks.

Code or HTTP Excerpt

```
      85 | @app.route("/logout")
      86 | def logout():
>>> 87 |     next_url = request.args.get("next", url_for("index"))
      88 |     response = redirect(next_url)
      89 |     response.delete_cookie("bank_uid")
```

Business Impact

Attackers could use your application to redirect users to phishing sites, potentially stealing credentials.

Fix Recommendation

Rule-based recommendation: Validate redirect URLs against a whitelist of allowed domains. Use relative paths instead of full URLs for internal redirects. Reject any redirect URLs pointing to external domains.

LLM recommendation: Validate redirect URLs against a whitelist of allowed domains. Use relative paths instead of full URLs for internal redirects. Reject any redirect URLs pointing to external domains.

[MEDIUM] #7: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:313

Risk Score	0.4508
------------	--------

Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:313
Vulnerability Type	UNKNOWN
CWE ID	CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
311 | cmd = request.args.get("cmd", "status")
312 | shell_cmd = f"echo Reconciled ledger for {cmd}"
>>> 313 | proc = subprocess.run(shell_cmd, shell=True, capture_output=True, text=True)
314 | summary = fetch_one(
315 |     "SELECT COUNT(*) AS users, (SELECT COUNT(*) FROM accounts) AS accounts, (SELECT COUNT(*)
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[MEDIUM] #8: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\laccount.html:24

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\laccount.html:24
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation

Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```
22 | {% for tx in transactions %}
23 | <tr>
>>> 24 | <td>{{ tx.counterparty|safe }}</td>
25 | <td>{{ '%.2F'|format(tx.amount) }}</td>
26 | <td>{{ tx.note|safe }}</td>
```

Business Impact

An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[MEDIUM] #9: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\dashboard.html:61

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\dashboard.html:61
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation
Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```
>>> 59 |         <td>{{ tx.counterparty }}</td>
      60 |         <td>{{ '%.2f'|format(tx.amount) }}</td>
      61 |         <td>{{ tx.note|safe }}</td>
      62 |     </tr>
      63 | {% endfor %}
```

Business Impact
An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[MEDIUM] #10: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\reflect.html:7

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\reflect.html:7
Vulnerability Type	XSS

CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation
 Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```

5 | <h1>Customer Notice Board</h1>
6 | <p class="subtle">This page intentionally renders query params unsafely.</p>
>>> 7 | <div class="alert">Hello {{ name|safe }}, message: {{ message|safe }}</div>
8 | <form method="get" class="form inline" action="/reflect">
9 | <label>Name<input type="text" name="name" value="{{ name|safe }}"></label>
```

Business Impact
 An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation
Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[MEDIUM] #11: Xss in C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\templates\support.html:23

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\templates\support.html:23
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation
 Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```

21 | <td>{{ ticket.name }}</td>
22 | <td>{{ ticket.subject }}</td>
>>> 23 | <td>{{ ticket.body|safe }}</td>
24 | </tr>
25 | {% endfor %}
```

Business Impact
 An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[LOW] #12: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:248

Risk Score	0.3384
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:248
Vulnerability Type	UNKNOWN
CWE ID	CWE-939: Improper Authorization in Handler for Custom URL Scheme
Detected By	semgrep

Explanation
A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
246 | def fetch_remote():
247 |     target = request.args.get("url", "http://127.0.0.1:5000/")
>>> 248 |     with urllib.request.urlopen(target, timeout=3) as handle:
249 |         body = handle.read(3000).decode("utf-8", errors="replace")
250 |     return Response(body, mimetype="text/plain")
```

Business Impact
This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[LOW] #13: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:369

Risk Score	0.2820
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:369
Vulnerability Type	UNKNOWN
CWE ID	CWE-489: Active Debug Code
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
367 |  
368 | if __name__ == "__main__":  
>>> 369 |     app.run(host="127.0.0.1", port=5001, debug=True)
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

Appendix

Deduplication Statistics

Total raw findings: 38 | After deduplication: 13 | Reduction: 66%

Scan Metadata

Scan ID	4959f563-beda-407d-9e79-2ba2e25dc376
Target	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab
Timestamp	2026-05-08T02:37:11.447295+00:00
Duration	17.0s
Scanners Used	semgrep
Scanner Versions	semgrep=1.159.0

Raw Findings Data (JSON)

```
[
  {
    "id": "09fe65a8-9f22-413a-b85c-9712834885d3",
    "title": "Insecure Deserialization in C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 282,
    "line_end": 282,
    "severity": "HIGH",
    "vuln_type": "INSECURE_DESERIALIZATION",
    "description": "Detected the use of an insecure deserialization library in a Flask route. These libraries are not safe for untrusted data.",
    "rule_id": "python.flask.security.insecure-deserialization.insecure-deserialization",
    "confidence": 0.9,
    "cwe_id": "CWE-502: Deserialization of Untrusted Data",
    "cvss_score": null,
    "code_snippet": "
      280 |         try:
      281 |             decoded = base64.b64decode(payload)
      282 |
",
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.6076,
    "risk_category": "HIGH",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Insecure deserialization occurs when untrusted data is deserialized without validation.",
    "fix_suggestion": "Validate and whitelist deserialized types. Avoid using pickle or yaml.load() with untrusted data.",
    "business_impact": "An attacker could execute arbitrary code on the server or manipulate application state.",
    "raw": {
      "check_id": "python.flask.security.insecure-deserialization.insecure-deserialization",
      "path": "C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
      "start": {
        "line": 282,
        "col": 26,
        "offset": 9736
      },
      "end": {
        "line": 282,
        "col": 47,
        "offset": 9757
      },
      "extra": {
```

```

    "message": "Detected the use of an insecure deserialization library in a Flask route. These libraries are insecure and can be exploited to compromise application security and potentially leak sensitive data.",
    "metadata": {
      "owasp": [
        "A08:2017 - Insecure Deserialization",
        "A08:2021 - Software and Data Integrity Failures",
        "A08:2025 - Software or Data Integrity Failures"
      ],
      "cwe": [
        "CWE-502: Deserialization of Untrusted Data"
      ],
      "references": [
        "https://docs.python.org/3/library/pickle.html"
      ],
      "category": "security",
      "technology": [
        "flask"
      ],
      "cwe2022-top25": true,
      "cwe2021-top25": true,
      "subcategory": [
        "audit"
      ],
      "likelihood": "LOW",
      "impact": "MEDIUM",
      "confidence": "LOW",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "Insecure Deserialization "
      ],
      "source": "https://semgrep.dev/r/python.flask.security.insecure-deserialization.insecure-deserialization",
      "shortlink": "https://sg.run/N45z"
    },
    "severity": "ERROR",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
}
},
{
  "id": "d4bf7ea3-b7d6-4f3a-9387-59bdd650bd48",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:70",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 70,
  "line_end": 70,
  "severity": "HIGH",
  "vuln_type": "UNKNOWN",
  "description": "Detected user input used to manually construct a SQL string. This is usually bad practice and can be exploited to compromise application security and potentially leak sensitive data.",
  "rule_id": "python.flask.security.injection.tainted-sql-string.tainted-sql-string",
  "confidence": 0.9,
  "cwe_id": "CWE-704: Incorrect Type Conversion or Cast",
  "cvss_score": null,
  "code_snippet": "
68 |         email = request.form.get('email', '')\n
69 |         password = request.form.get('password', '')\n
",
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.539,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be exploited to compromise application security and potentially leak sensitive data.",
  "fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelines for more information.",
  "business_impact": "This vulnerability could be exploited to compromise application security and potentially leak sensitive data.",
  "raw": {
    "check_id": "python.flask.security.injection.tainted-sql-string.tainted-sql-string",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "start": {
      "line": 70,
      "col": 17,
      "offset": 2086
    }
  }
}

```

```

    },
    "end": {
        "line": 70,
        "col": 91,
        "offset": 2160
    },
    "extra": {
        "message": "Detected user input used to manually construct a SQL string. This is usually bad practice",
        "metadata": {
            "cwe": [
                "CWE-704: Incorrect Type Conversion or Cast"
            ],
            "owasp": [
                "A01:2017 - Injection",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "references": [
                "https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-textual-sql",
                "https://www.tutorialspoint.com/sqlalchemy/sqlalchemy_quick_guide.htm",
                "https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-more-specific-text-with-table-expressions"
            ],
            "category": "security",
            "technology": [
                "sqlalchemy",
                "flask"
            ],
            "subcategory": [
                "vuln"
            ],
            "impact": "MEDIUM",
            "likelihood": "MEDIUM",
            "confidence": "MEDIUM",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [
                "Improper Validation"
            ],
            "source": "https://semgrep.dev/r/python.flask.security.injection.tainted-sql-string.tainted-sql-string",
            "shortlink": "https://sg.run/JxZj"
        },
        "severity": "ERROR",
        "fingerprint": "requires login",
        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "81603de3-fe37-4f62-bdee-b042d6858c72",
    "title": "Sql Injection in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 68,
    "line_end": 81,
    "severity": "MEDIUM",
    "vuln_type": "SQL_INJECTION",
    "description": "User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection.",
    "rule_id": "python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute",
    "confidence": 0.7,
    "cwe_id": "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')",
    "cvss_score": null,
    "code_snippet": "
        66 |         error = None\n
        67 |         if request.method == \"POST\":\n
        68 |             data = request.get_json()\n
        69 |             if data is not None:\n
        70 |                 username = data.get('username')\n
        71 |                 password = data.get('password')\n
        72 |                 if username and password:\n
        73 |                     user = User.query.filter_by(username=username).first()\n
        74 |                     if user and user.password == password:\n
        75 |                         session['username'] = username\n
        76 |                         return redirect(url_for('index'))\n
        77 |                     else:\n
        78 |                         error = 'Invalid username or password'\n
        79 |             return render_template('login.html', error=error)\n
        80 |     return render_template('login.html', error=error)\n
        81 |
    ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.517,
    "risk_category": "HIGH",
    "category_source": "vuln_floor",
    "category_reason": "minimum floor for SQL_INJECTION",
    "llm_explanation": "SQL Injection occurs when user-controlled input is directly concatenated into SQL queries. In Python, use placeholders for user input instead of string concatenation.",
    "fix_suggestion": "Use parameterized queries instead of string concatenation. In Python, use placeholders for user input instead of string concatenation."
}

```

```

"business_impact": "An attacker could read, modify, or delete all data in the database, potentially exposing sensitive information.",
"raw": {
  "check_id": "python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection",
  "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "start": {
    "line": 68,
    "col": 9,
    "offset": 1978
  },
  "end": {
    "line": 81,
    "col": 51,
    "offset": 2624
  },
  "extra": {
    "message": "User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection.",
    "metadata": {
      "cwe": [
        "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')"
      ],
      "owasp": [
        "A01:2017 - Injection",
        "A03:2021 - Injection",
        "A05:2025 - Injection"
      ],
      "references": [
        "https://docs.djangoproject.com/en/3.0/topics/security/#sql-injection-protection"
      ],
      "category": "security",
      "technology": [
        "django"
      ],
      "cwe2022-top25": true,
      "cwe2021-top25": true,
      "subcategory": [
        "vuln"
      ],
      "likelihood": "MEDIUM",
      "impact": "HIGH",
      "confidence": "MEDIUM",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "SQL Injection"
      ],
      "source": "https://semgrep.dev/r/python.django.security.injection.sql.sql-injection-using-db-cursor-execute",
      "shortlink": "https://sg.run/qx7y"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
},
{
  "id": "6ffef9ac-1bc1-49bd-b1d0-932d987ea333",
  "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\admin.html",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\admin.html",
  "line_start": 22,
  "line_end": 22,
  "severity": "MEDIUM",
  "vuln_type": "XSS",
  "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False'. This could lead to a Cross-site Scripting (XSS) attack.",
  "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
  "confidence": 0.7,
  "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
  "cvss_score": null,
  "code_snippet": "
20 |         <button class='btn primary' type='submit'>Run</button>
21 |     "
  "sources": [
    "semgrep"
  ]
}

```

```

],
"risk_score": 0.4747,
"risk_category": "MEDIUM",
"category_source": "score",
"category_reason": "score-threshold",
"llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output",
"fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in HTML",
"business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or perform other unauthorized actions",
"raw": {
  "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
  "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\admin.html",
  "start": {
    "line": 22,
    "col": 26,
    "offset": 923
  },
  "end": {
    "line": 22,
    "col": 51,
    "offset": 948
  },
  "extra": {
    "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with `render`",
    "metadata": {
      "cwe": [
        "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')"
      ],
      "owasp": [
        "A07:2017 - Cross-Site Scripting (XSS)",
        "A03:2021 - Injection",
        "A05:2025 - Injection"
      ],
      "references": [
        "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
      ],
      "category": "security",
      "technology": [
        "flask"
      ],
      "cwe2022-top25": true,
      "cwe2021-top25": true,
      "subcategory": [
        "audit"
      ],
      "likelihood": "LOW",
      "impact": "MEDIUM",
      "confidence": "LOW",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "Cross-Site-Scripting (XSS)"
      ],
      "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
      "shortlink": "https://sg.run/W8og"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
}
},
{
  "id": "ecf7ec9b-24e6-446e-8fa3-7c32586bf29a",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:10",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 10,
  "line_end": 10,
  "severity": "HIGH",
  "vuln_type": "UNKNOWN",
  "description": "The Python documentation recommends using `defusedxml` instead of `xml` because the native `xml` module is vulnerable to a Denial of Service (DoS) attack. The attack is triggered by parsing a maliciously crafted XML document that causes the parser to consume excessive memory and CPU resources, eventually leading to a system crash or service outage."
}

```



```

"rule_id": "python.lang.security.use-defused-xml.use-defused-xml",
"confidence": 0.9,
"cwe_id": "CWE-611: Improper Restriction of XML External Entity Reference",
"cvss_score": null,
"code_snippet": "      8 | import subprocess\n                        9 | import urllib.request\n>>>    10 | import xml.etree.ElementTree as ET\n\n    sources: [\n        \"semgrep\"\n    ],\n    risk_score: 0.4508,\n    risk_category: \"MEDIUM\",\n    category_source: \"score\",\n    category_reason: \"score-threshold\",\n    llm_explanation: \"A security vulnerability was detected in your code. This issue could potentially be exploited to compromise application security and potentially leak sensitive data. Review the flagged code and apply security best practices. Consult the OWASP guidelines for more information on XML External Entity (XXE) attacks.\",\n    fix_suggestion: \"Review the flagged code and apply security best practices. Consult the OWASP guidelines for more information on XML External Entity (XXE) attacks.\",\n    business_impact: \"This vulnerability could be exploited to compromise application security and potentially leak sensitive data. Review the flagged code and apply security best practices. Consult the OWASP guidelines for more information on XML External Entity (XXE) attacks.\",\n    raw: {\n        \"check_id\": \"python.lang.security.use-defused-xml.use-defused-xml\",\n        \"path\": \"C:\\\\Users\\\\Dhruv\\\\Desktop\\\\CyberSecurity Project\\\\Test\\\\vuln_bank_lab\\\\app.py\",\n        \"start\": {\n            \"line\": 10,\n            \"col\": 1,\n            \"offset\": 148\n        },\n        \"end\": {\n            \"line\": 10,\n            \"col\": 35,\n            \"offset\": 182\n        },\n        \"extra\": {\n            \"message\": \"The Python documentation recommends using `defusedxml` instead of `xml` because the native `xml` module is vulnerable to XML External Entity (XXE) attacks.\",\n            \"metadata\": {\n                \"owasp\": [\n                    \"A04:2017 - XML External Entities (XXE)\",\n                    \"A05:2021 - Security Misconfiguration\",\n                    \"A02:2025 - Security Misconfiguration\"\n                ],\n                \"cwe\": [\n                    \"CWE-611: Improper Restriction of XML External Entity Reference\"\n                ],\n                \"references\": [\n                    \"https://docs.python.org/3/library/xml.html\",\n                    \"https://github.com/tiran/defusedxml\",\n                    \"https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processing\"\n                ],\n                \"category\": \"security\",\n                \"technology\": [\n                    \"python\"\n                ],\n                \"cwe2022-top25\": true,\n                \"cwe2021-top25\": true,\n                \"subcategory\": [\n                    \"audit\"\n                ],\n                \"likelihood\": \"LOW\",\n                \"impact\": \"MEDIUM\",\n                \"confidence\": \"LOW\",\n                \"license\": \"Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license\",\n                \"vulnerability_class\": [\n                    \"XML Injection\"\n                ],\n                \"source\": \"https://semgrep.dev/r/python.lang.security.use-defused-xml.use-defused-xml\",\n                \"shortlink\": \"https://sg.run/kX47\"\n            }\n        },\n        \"severity\": \"ERROR\",\n        \"fingerprint\": \"requires login\",\n        \"lines\": \"requires login\",\n        \"validation_state\": \"NO_VALIDATOR\",\n        \"engine_kind\": \"OSS\"\n    }\n}\n}

```

```

{
  "id": "aee60299-1556-46f7-9dfc-c12c5b070302",
  "title": "Open Redirect in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 87,
  "line_end": 88,
  "severity": "HIGH",
  "vuln_type": "OPEN_REDIRECT",
  "description": "Data from request is passed to redirect(). This is an open redirect and could be exploited",
  "rule_id": "python.flask.security.open-redirect.open-redirect",
  "confidence": 0.9,
  "cwe_id": "CWE-601: URL Redirection to Untrusted Site ('Open Redirect')",
  "cvss_score": null,
  "code_snippet": "
      85 | @app.route(\"/logout\")\n
      86 | def logout():\n>>> 87 |     next_url =
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.4508,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "Open redirect vulnerabilities allow attackers to redirect users to malicious websites",
  "fix_suggestion": "Validate redirect URLs against a whitelist of allowed domains. Use relative paths instead of absolute paths",
  "business_impact": "Attackers could use your application to redirect users to phishing sites, potentially stealing sensitive information",
  "raw": {
    "check_id": "python.flask.security.open-redirect.open-redirect",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "start": {
      "line": 87,
      "col": 5,
      "offset": 2756
    },
    "end": {
      "line": 88,
      "col": 34,
      "offset": 2844
    },
    "extra": {
      "message": "Data from request is passed to redirect(). This is an open redirect and could be exploited",
      "metadata": {
        "cwe": [
          "CWE-601: URL Redirection to Untrusted Site ('Open Redirect')"
        ],
        "owasp": [
          "A01:2021 - Broken Access Control",
          "A01:2025 - Broken Access Control"
        ],
        "references": [
          "https://flask-login.readthedocs.io/en/latest/#login-example",
          "https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html",
          "https://docs.python.org/3/library/urllib.parse.html#url-parsing"
        ],
        "category": "security",
        "technology": [
          "flask"
        ],
        "subcategory": [
          "audit"
        ],
        "likelihood": "LOW",
        "impact": "MEDIUM",
        "confidence": "LOW",
        "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
        "vulnerability_class": [
          "Open Redirect"
        ],
        "source": "https://semgrep.dev/r/python.flask.security.open-redirect.open-redirect",
        "shortlink": "https://sg.run/kXe2"
      },
      "severity": "ERROR",
      "fingerprint": "requires login",

```

```

        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "8823c6d2-354c-482f-9282-6cb94f5016e3",
    "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py:313",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 313,
    "line_end": 313,
    "severity": "HIGH",
    "vuln_type": "UNKNOWN",
    "description": "Detected user input entering a `subprocess` call unsafely. This could result in a command execution.",
    "rule_id": "python.flask.security.injection.subprocess-injection.subprocess-injection",
    "confidence": 0.9,
    "cwe_id": "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')",
    "cvss_score": null,
    "code_snippet": "
        311 |         cmd = request.args.get(\"cmd\", \"status\")\n
        312 |         shell_cmd =
    ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4508,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be exploited to compromise application security and potentially execute arbitrary commands on the system.",
    "fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelines for command injection prevention.",
    "business_impact": "This vulnerability could be exploited to compromise application security and potentially execute arbitrary commands on the system.",
    "raw": {
        "check_id": "python.flask.security.injection.subprocess-injection.subprocess-injection",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
        "start": {
            "line": 313,
            "col": 12,
            "offset": 11098
        },
        "end": {
            "line": 313,
            "col": 81,
            "offset": 11167
        }
    },
    "extra": {
        "message": "Detected user input entering a `subprocess` call unsafely. This could result in a command execution.",
        "metadata": {
            "category": "security",
            "technology": [
                "flask"
            ],
            "owasp": [
                "A01:2017 - Injection",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "cwe": [
                "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')"
            ],
            "references": [
                "https://semgrep.dev/docs/cheat-sheets/python-command-injection/"
            ],
            "confidence": "HIGH",
            "cwe2022-top25": true,
            "cwe2021-top25": true,
            "subcategory": [
                "vuln"
            ],
            "likelihood": "HIGH",
            "impact": "MEDIUM",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [

```

```

        "Command Injection"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.injection.subprocess-injection.subprocess-",
    "shortlink": "https://sg.run/5gw3"
},
"severity": "ERROR",
"fingerprint": "requires login",
"lines": "requires login",
"validation_state": "NO_VALIDATOR",
"engine_kind": "OSS"
}
}
},
{
    "id": "87718e34-ea58-4e66-b7a9-9198b0e12afc",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\account.html",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\account.html",
    "line_start": 24,
    "line_end": 24,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False'.",
    "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "
                22 |         {% for tx in transactions %}
                23 |             <tr>
                24 |
",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4042,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper escaping.",
    "fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in HTML.",
    "business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or perform other unauthorized actions.",
    "raw": {
        "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\account.html",
        "start": {
            "line": 24,
            "col": 13,
            "offset": 667
        },
        "end": {
            "line": 24,
            "col": 39,
            "offset": 693
        },
        "extra": {
            "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False'.",
            "metadata": {
                "cwe": [
                    "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')"
                ],
                "owasp": [
                    "A07:2017 - Cross-Site Scripting (XSS)",
                    "A03:2021 - Injection",
                    "A05:2025 - Injection"
                ],
                "references": [
                    "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
                ],
                "category": "security",
                "technology": [
                    "flask"
                ],
                "cwe2022-top25": true,
                "cwe2021-top25": true,
                "subcategory": [

```

```

        "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
        "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.tem",
    "shortlink": "https://sg.run/W8og"
},
{
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
}
}
},
{
    "id": "f41cc074-0aab-40d3-ab53-3636b0f31264",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\dashb",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\dashboard.htm",
    "line_start": 61,
    "line_end": 61,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with '
    "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "
        59 |
        <td>{{ tx.counterparty }}</td>\n
        60 |
        <td>\u20b9{{
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4042,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output",
    "fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in H",
    "business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or p",
    "raw": {
        "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\dashboard.f",
        "start": {
            "line": 61,
            "col": 15,
            "offset": 2039
        },
        "end": {
            "line": 61,
            "col": 33,
            "offset": 2057
        },
        "extra": {
            "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with '
            "metadata": {
                "cwe": [
                    "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
                ],
                "owasp": [
                    "A07:2017 - Cross-Site Scripting (XSS)",
                    "A03:2021 - Injection",
                    "A05:2025 - Injection"
                ],
                "references": [
                    "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
                ],
            },
        },
    },

```

```

"category": "security",
"technology": [
  "flask"
],
"cwe2022-top25": true,
"cwe2021-top25": true,
"subcategory": [
  "audit"
],
"likelihood": "LOW",
"impact": "MEDIUM",
"confidence": "LOW",
"license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
"vulnerability_class": [
  "Cross-Site-Scripting (XSS)"
],
"source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
"shortlink": "https://sg.run/W8og"
},
"severity": "WARNING",
"fingerprint": "requires login",
"lines": "requires login",
"validation_state": "NO_VALIDATOR",
"engine_kind": "OSS"
}
}
},
{
  "id": "62983325-ce6b-472f-a0d1-7c3567dc64b4",
  "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\reflect.html",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\reflect.html",
  "line_start": 7,
  "line_end": 7,
  "severity": "MEDIUM",
  "vuln_type": "XSS",
  "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False' in the render function.",
  "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
  "confidence": 0.7,
  "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
  "cvss_score": null,
  "code_snippet": "5 | <h1>Customer Notice Board</h1>\n6 | <p class=\"subtle\">This page is under construction.</p>",
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.4042,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper escaping.",
  "fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in HTML.",
  "business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or deface the website.",
  "raw": {
    "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\reflect.html",
    "start": {
      "line": 7,
      "col": 28,
      "offset": 255
    },
    "end": {
      "line": 7,
      "col": 43,
      "offset": 270
    }
  },
  "extra": {
    "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False' in the render function.",
    "metadata": {
      "cwe": [
        "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')"
      ],
      "owasp": [

```

```

        "A07:2017 - Cross-Site Scripting (XSS)",
        "A03:2021 - Injection",
        "A05:2025 - Injection"
    ],
    "references": [
        "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
    ],
    "category": "security",
    "technology": [
        "flask"
    ],
    "cwe2022-top25": true,
    "cwe2021-top25": true,
    "subcategory": [
        "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
        "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "shortlink": "https://sg.run/W8og"
},
{
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
}
},
{
    "id": "cab71e1c-4e9f-43b3-a4a6-f2b624ace6b1",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\support.html",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\support.html",
    "line_start": 23,
    "line_end": 23,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False'. This is a security vulnerability as it allows for Cross-Site Scripting (XSS) attacks.",
    "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "
21 |         <td>{{ ticket.name }}</td>\n
22 |         <td>{{ ticket.sub
",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4042,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper escaping. In this case, the Flask template engine is configured with 'autoescape=False', which allows raw HTML and JavaScript to be rendered directly. This can lead to XSS attacks if the data being rendered is user-controlled.",
    "fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in the template. Ensure 'autoescape=True' is set in the Flask template engine configuration.",
    "business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or perform other actions that compromise the security and integrity of the application.",
    "raw": {
        "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\support.html",
        "start": {
            "line": 23,
            "col": 15,
            "offset": 853
        },
        "end": {
            "line": 23,
            "col": 37,
            "offset": 875
        }
    },
},

```

```

"extra": {
  "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with '
  "metadata": {
    "cwe": [
      "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    ],
    "owasp": [
      "A07:2017 - Cross-Site Scripting (XSS)",
      "A03:2021 - Injection",
      "A05:2025 - Injection"
    ],
    "references": [
      "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
    ],
    "category": "security",
    "technology": [
      "flask"
    ],
    "cwe2022-top25": true,
    "cwe2021-top25": true,
    "subcategory": [
      "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
      "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.tem
    "shortlink": "https://sg.run/W8og"
  },
  "severity": "WARNING",
  "fingerprint": "requires login",
  "lines": "requires login",
  "validation_state": "NO_VALIDATOR",
  "engine_kind": "OSS"
}
}
},
{
  "id": "6d5cf248-8409-4262-8848-4926879e347b",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:248",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 248,
  "line_end": 248,
  "severity": "MEDIUM",
  "vuln_type": "UNKNOWN",
  "description": "Detected a dynamic value being used with urllib. urllib supports 'file://' schemes, so a
  "rule_id": "python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected",
  "confidence": 0.7,
  "cwe_id": "CWE-939: Improper Authorization in Handler for Custom URL Scheme",
  "cvss_score": null,
  "code_snippet": "
    246 | def fetch_remote():\n
    247 |     target = request.args.get(\"url\", \"ht
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.3384,
  "risk_category": "LOW",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be
  "fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelin
  "business_impact": "This vulnerability could be exploited to compromise application security and potent
  "raw": {
    "check_id": "python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "start": {
      "line": 248,
      "col": 10,

```



```

        "offset": 8376
    },
    "end": {
        "line": 248,
        "col": 51,
        "offset": 8417
    },
    "extra": {
        "message": "Detected a dynamic value being used with urllib. urllib supports 'file://\' schemes, so a",
        "metadata": {
            "cwe": [
                "CWE-939: Improper Authorization in Handler for Custom URL Scheme"
            ],
            "owasp": "A01:2017 - Injection",
            "source-rule-url": "https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee954c2b1",
            "bandit-code": "B310",
            "asvs": {
                "control_id": "5.2.4 Dynamic Code Execution Features",
                "control_url": "https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-Sanitization",
                "section": "V5: Validation, Sanitization and Encoding Verification Requirements",
                "version": "4"
            },
            "category": "security",
            "technology": [
                "python"
            ],
            "references": [
                "https://cwe.mitre.org/data/definitions/939.html"
            ],
            "subcategory": [
                "audit"
            ],
            "likelihood": "LOW",
            "impact": "LOW",
            "confidence": "LOW",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [
                "Improper Authorization"
            ],
            "source": "https://semgrep.dev/r/python.lang.security.audit.dynamic-urllib-use-detected.dynamic-u",
            "shortlink": "https://sg.run/dKZZ"
        },
        "severity": "WARNING",
        "fingerprint": "requires login",
        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "c6c20b77-a222-4560-931e-92a3fc47e31b",
    "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberSecurity Project\\Test\\vuln_bank_lab\\app.py:369",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberSecurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 369,
    "line_end": 369,
    "severity": "MEDIUM",
    "vuln_type": "UNKNOWN",
    "description": "Detected Flask app with debug=True. Do not deploy to production with this flag enabled a",
    "rule_id": "python.flask.security.audit.debug-enabled.debug-enabled",
    "confidence": 0.7,
    "cwe_id": "CWE-489: Active Debug Code",
    "cvss_score": null,
    "code_snippet": "
        367 | \n
        368 | if __name__ == \"__main__\":\n>>>
        369 |         app.run(host=\"127.0.0.1\", port=5000)
    ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.282,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",

```

```

"llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be
"fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelin
"business_impact": "This vulnerability could be exploited to compromise application security and potent
"raw": {
  "check_id": "python.flask.security.audit.debug-enabled.debug-enabled",
  "path": "C:\\Users\\Dhruv\\Desktop\\CyberSecurity Project\\Test\\vuln_bank_lab\\app.py",
  "start": {
    "line": 369,
    "col": 5,
    "offset": 12789
  },
  "end": {
    "line": 369,
    "col": 53,
    "offset": 12837
  },
  "extra": {
    "message": "Detected Flask app with debug=True. Do not deploy to production with this flag enabled a
    "metadata": {
      "cwe": [
        "CWE-489: Active Debug Code"
      ],
      "owasp": "A06:2017 - Security Misconfiguration",
      "references": [
        "https://labs.detectify.com/2015/10/02/how-patreon-got-hacked-publicly-exposed-werkzeug-debugger
      ],
      "category": "security",
      "technology": [
        "flask"
      ],
      "subcategory": [
        "vuln"
      ],
      "likelihood": "HIGH",
      "impact": "MEDIUM",
      "confidence": "HIGH",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "Active Debug Code"
      ],
      "source": "https://semgrep.dev/r/python.flask.security.audit.debug-enabled.debug-enabled",
      "shortlink": "https://sg.run/dKrd"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
}
}
]

```